EJERCICIOS BLOQUE 3 NIVEL MEDIO

Bloque 1: Red y conectividad (1–7)

Ejercicio 1. Ping con análisis

Haz ping 4 veces a un host indicado por el usuario y muestra la latencia mínima, máxima y media.

```
$host = Read-Host "Introduce un host"

$datos = Test-Connection $host -Count 4

[pscustomobject]@{

    Host = $host

    Min = ($datos | Measure-Object ResponseTime -Minimum).Minimum

    Max = ($datos | Measure-Object ResponseTime -Maximum).Maximum

    Media = [math]::Round(($datos | Measure-Object ResponseTime -Average).Average, 1)

}
```

Ejercicio 2. Ping a varios hosts desde una lista

```
PS C:\Windows\System32> echo "127.0.0.1"> "c:\hosts.txt"
PS C:\Windows\System32> $hosts = Get-Content "C:\hosts.txt"
>> foreach ($h in $hosts) {
>>     try {
>>         $media = (Test-Connection $h -Count 4 -ErrorAction Stop | Measure-Object ResponseTime -Average).Average
>>         "{0}: {1:N1} ms" -f $h, $media
>>     } catch {
>>         "{0}: sin respuesta" -f $h
>>     }
>> }
>>
127.0.0.1:  ms
PS C:\Windows\System32>
```

Lee una lista de hosts desde un archivo hosts.txt y muestra el tiempo medio de respuesta de cada uno.

```
$hosts = Get-Content "C:\hosts.txt"

foreach ($h in $hosts) {

    try {

        $media = (Test-Connection $h -Count 4 -ErrorAction Stop | Measure-Object ResponseTime -Average).Average

        "{0}: {1:N1} ms" -f $h, $media
```

```
    } catch {
        "{0}: sin respuesta" -f $h
    }
}
```

Ejercicio 3. Comprobación de puertos comunes

Verifica si un servidor responde en los puertos 80 y 443.

```
$host = Read-Host "Servidor"
foreach ($p in 80,443) {
    $resultado = Test-NetConnection -ComputerName $host -Port $p
    "{0}: puerto {1} -> {2}" -f $host, $p, ($resultado.TcpTestSucceeded ? "Abierto" : "Cerrado")
}
```

Ejercicio 4. Ping con resultados en CSV

```
PS C:\Windows\System32> $hosts = "8.8.8.8","1.1.1.1","github.com"
>> $resultados = foreach ($h in $hosts) {
>>     $media = (Test-Connection $h -Count 4 | Measure-Object ResponseTime -Average).Average
>>     [pscustomobject]@{ Host=$h; Media=[math]::Round($media,1) }
>> }
>> $resultados | Export-Csv "C:\ping_resultados.csv" -NoTypeInformation
>>
PS C:\Windows\System32> cat C:\ping_resultados.csv
"Host","Media"
"8.8.8.8","0"
"1.1.1.1","0"
"github.com","0"
PS C:\Windows\System32>
```

Haz ping a 3 servidores y guarda el promedio de latencia en un CSV.

```
$hosts = "8.8.8.8","1.1.1.1","github.com"
$resultados = foreach ($h in $hosts) {
    $media = (Test-Connection $h -Count 4 | Measure-Object ResponseTime -Average).Average
    [pscustomobject]@{ Host=$h; Media=[math]::Round($media,1) }
}
$resultados | Export-Csv "C:\Scripts\ping_resultados.csv" -NoTypeInformation
```

Ejercicio 5. Adaptadores activos

```
PS C:\Windows\System32> Get-NetAdapter | Where-Object Status -eq 'Up' | Sort-Object LinkSpeed -Descending | Select Name, LinkSpee
d

Name            LinkSpeed
----            ---------
vEthernet (WSL) 10 Gbps
Ethernet 3      1 Gbps
Ethernet 2      1 Gbps

PS C:\Windows\System32>
```

Muestra solo los adaptadores con estado “Up”, ordenados por velocidad.

Get-NetAdapter | Where-Object Status -eq 'Up' | Sort-Object LinkSpeed -Descending | Select Name, LinkSpeed

---

Ejercicio 6. Detectar IP pública

```
PS C:\Windows\System32> (Invoke-RestMethod -Uri "https://api.ipify.org?format=json").ip
2.136.142.51
PS C:\Windows\System32>
```

Obtén y muestra la dirección IP pública del equipo.

(Invoke-RestMethod -Uri "https://api.ipify.org?format=json").ip

---

Ejercicio 7. Diagnóstico completo de red

```
PS C:\Windows\System32> $ip = (Get-NetIPAddress -AddressFamily IPv4 | Where-Object {$_.IPAddress -notlike '169.*'} | Select-Objec
t -First 1 -ExpandProperty IPAddress)
>> $gateway = (Get-NetRoute | Where-Object DestinationPrefix -eq '0.0.0.0/0').NextHop
>> $dns = (Get-DnsClientServerAddress -AddressFamily IPv4).ServerAddresses -join ", "
>> $ping = (Test-Connection 8.8.8.8 -Count 4 | Measure-Object ResponseTime -Average).Average
>> @"
>> IP: $ip
>> Puerta de enlace: $gateway
>> DNS: $dns
>> Latencia 8.8.8.8: $([math]::Round($ping,1)) ms
>> "@ | Out-File "C:\ diagnostico.txt"
>>
PS C:\Windows\System32> cat "C:\ diagnostico.txt"
IP: 172.22.176.1
Puerta de enlace: 192.168.0.1
DNS: 80.58.61.254, 80.58.61.250
Latencia 8.8.8.8: 0 ms
PS C:\Windows\System32>
```

Guarda en un archivo TXT: IP local, puerta de enlace, DNS y latencia media a 8.8.8.8.

$ip = (Get-NetIPAddress -AddressFamily IPv4 | Where-Object {$_.IPAddress -notlike '169.*'} | Select-Object -First 1 -ExpandProperty IPAddress)

$gateway = (Get-NetRoute | Where-Object DestinationPrefix -eq '0.0.0.0/0').NextHop

$dns = (Get-DnsClientServerAddress -AddressFamily IPv4).ServerAddresses -join ", "

$ping = (Test-Connection 8.8.8.8 -Count 4 | Measure-Object ResponseTime -Average).Average

@"

IP: $ip

Puerta de enlace: $gateway

DNS: $dns

Latencia 8.8.8.8: $([math]::Round($ping,1)) ms

"@ | Out-File "C:\ diagnostico.txt"

🔹 Bloque 2: Sistema y procesos (8–12)

Ejercicio 8. Procesos que más CPU consumen

```
PS C:\Windows\System32> Get-Process | Sort-Object CPU -Descending | Select-Object -First 5 Name, CPU

Name                      CPU
----                      ---
VirtualBoxVM        13638,19
System               3874,95
svchost               882,08
Memory Compression    771,53
svchost               233,55

PS C:\Windows\System32>
```

Muestra los 5 procesos con más uso de CPU.

Get-Process | Sort-Object CPU -Descending | Select-Object -First 5 Name, CPU

Ejercicio 9. Buscar un proceso por nombre

```
PS C:\Windows\System32> $nombre = Read-Host "Introduce parte del nombre del proceso"
>> Get-Process | Where-Object Name -like "*$nombre*" | Select Name, Id
>>
Introduce parte del nombre del proceso: firefox

Name       Id
----       --
firefox  5380
firefox  6880
firefox  7316
firefox  8220
firefox  8300
firefox  9652
firefox 10360
firefox 10404
firefox 10432
firefox 11528
firefox 11700
firefox 12692
firefox 12796
firefox 13480
firefox 15304

PS C:\Windows\System32>
```

Pide al usuario un nombre parcial y muestra los procesos que lo contengan.

$nombre = Read-Host "Introduce parte del nombre del proceso"

Get-Process | Where-Object Name -like "*$nombre*" | Select Name, Id

---

Ejercicio 10. Exportar servicios a CSV

```
PS C:\Windows\System32> Get-Service | Where-Object Status -eq "Stopped" |
>> Select-Object Name, StartType |
>> Export-Csv "C:\servicios_det.csv" -NoTypeInformation
>>
Get-Service: Service 'McpManagementService (McpManagementService)' cannot be queried due to the following error: PermissionDenied
PS C:\Windows\System32> cat "C:\servicios_det.csv"
"Name","StartType"
"AarSvc_9ac04efd","Manual"
"AJRouter","Manual"
"ALG","Manual"
"AnyDesk","Disabled"
"AppIDSvc","Manual"
"AppReadiness","Manual"
"AppVClient","Disabled"
"AssignedAccessManagerSvc","Manual"
"autotimesvc","Manual"
"AxInstSV","Manual"
"BcastDVRUserService_9ac04efd","Manual"
"BDESVC","Manual"
"BITS","Manual"
"BluetoothUserService_9ac04efd","Manual"
"BTAGService","Manual"
"bthserv","Manual"
"camsvc","Manual"
"CaptureService_9ac04efd","Manual"
"CertPropSvc","Manual"
"ClipSVC","Manual"
```

Guarda los servicios detenidos en un CSV con nombre y tipo de inicio.

Get-Service | Where-Object Status -eq "Stopped" |

Select-Object Name, StartType |

Export-Csv "C:\Scripts\servicios_det.csv" -NoTypeInformation

Ejercicio 11. Mostrar espacio libre en cada disco

```
PS C:\Windows\System32> Get-CimInstance Win32_LogicalDisk -Filter "DriveType=3" |
>> Select-Object DeviceID,
>> @{Name="Libre(GB)";Expression={[math]::Round($_.FreeSpace/1GB,2)}},
>> @{Name="Usado(%)";Expression={[math]::Round((1 - ($_.FreeSpace / $_.Size))*100,1)}}
>>

DeviceID Libre(GB) Usado(%)
-------- --------- --------
C:           11,30    94,90

PS C:\Windows\System32>
```

Muestra la unidad, espacio libre (GB) y porcentaje usado.

Get-CimInstance Win32_LogicalDisk -Filter "DriveType=3" |

Select-Object DeviceID,

@{Name="Libre(GB)";Expression={[math]::Round($_.FreeSpace/1GB,2)}},

@{Name="Usado(%)";Expression={[math]::Round((1 - ($_.FreeSpace / $_.Size))*100,1)}}

Ejercicio 12. Buscar archivos grandes

```
PS C:\Windows\System32> $ruta = Read-Host "Ruta a analizar"
>> Get-ChildItem -Path $ruta -Recurse -File | Sort-Object Length -Descending | Select-Object -First 10 Name, @{n='Tamaño(MB)';e={
[math]::Round($_.Length/1MB,2)}}
>>
Ruta a analizar: c:\usuarios
Get-ChildItem:
Line |
   2 |  Get-ChildItem -Path $ruta -Recurse -File | Sort-Object Length -Descen …
     |  ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
     | Access to the path 'C:\Program Files\Windows Defender Advanced Threat Protection\Classification\Configuration' is denied.
Get-ChildItem:
Line |
   2 |  Get-ChildItem -Path $ruta -Recurse -File | Sort-Object Length -Descen …
     |  ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
     | Access to the path 'C:\Windows\CSC' is denied.
PS C:\Windows\System32>
```

Lista los 10 archivos más grandes de una carpeta.

$ruta = Read-Host "Ruta a analizar"

Get-ChildItem -Path $ruta -Recurse -File | Sort-Object Length -Descending | Select-Object -First 10 Name, @{n='Tamaño(MB)';e={[math]::Round($_.Length/1MB,2)}}

🔹 Bloque 3: Eventos (13–16)

Ejercicio 13. Errores del sistema de la última hora

```
PS C:\Windows\System32> $desde = (Get-Date).AddHours(-1)
>> Get-WinEvent -FilterHashtable @{LogName='System'; Level=2; StartTime=$desde} |
>> Select TimeCreated, ProviderName, Message
>>
Get-WinEvent:
Line |
   2 |  Get-WinEvent -FilterHashtable @{LogName='System'; Level=2; StartTime= …
     |  ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
     | No events were found that match the specified selection criteria.
PS C:\Windows\System32>
```

$desde = (Get-Date).AddHours(-1)

Get-WinEvent -FilterHashtable @{LogName='System'; Level=2; StartTime=$desde} |

Select TimeCreated, ProviderName, Message

Ejercicio 14. Eventos de una fuente específica

```
PS C:\Windows\System32> Get-WinEvent -FilterHashtable @{LogName='System'; ProviderName='Service Control Manager'} -MaxEvents 10 |

>> Select TimeCreated, Id, Message
>>

TimeCreated           Id Message
-----------           -- -------
14/11/2025 11:00:21 7040 El tipo de inicio del servicio Servicio de transferencia inteligente en segundo plano (BITS) se cambió…
14/11/2025 10:58:17 7040 El tipo de inicio del servicio Servicio de transferencia inteligente en segundo plano (BITS) se cambió…
14/11/2025 8:37:43  7045 Se instaló un servicio en el sistema.…
14/11/2025 7:48:29  7040 El tipo de inicio del servicio Servicio de transferencia inteligente en segundo plano (BITS) se cambió…
14/11/2025 7:45:52  7040 El tipo de inicio del servicio Servicio de transferencia inteligente en segundo plano (BITS) se cambió…
13/11/2025 12:56:34 7040 El tipo de inicio del servicio Servicio de transferencia inteligente en segundo plano (BITS) se cambió…
13/11/2025 12:53:33 7040 El tipo de inicio del servicio Servicio de transferencia inteligente en segundo plano (BITS) se cambió…
12/11/2025 15:22:23 7040 El tipo de inicio del servicio Servicio de transferencia inteligente en segundo plano (BITS) se cambió…
12/11/2025 15:19:42 7040 El tipo de inicio del servicio Servicio de transferencia inteligente en segundo plano (BITS) se cambió…
12/11/2025 10:59:39 7040 El tipo de inicio del servicio Servicio de transferencia inteligente en segundo plano (BITS) se cambió…

PS C:\Windows\System32>
```

Muestra eventos del proveedor “Service Control Manager”.

Get-WinEvent -FilterHashtable @{LogName='System'; ProviderName='Service Control Manager'} -MaxEvents 10 |

Select TimeCreated, Id, Message

Ejercicio 15. Contar errores en las últimas 24h

```
PS C:\Windows\System32> $desde = (Get-Date).AddHours(-24)
>> (Get-WinEvent -FilterHashtable @{LogName='System'; Level=2; StartTime=$desde}).Count
>>
7
PS C:\Windows\System32>
```

$desde = (Get-Date).AddHours(-24)

(Get-WinEvent -FilterHashtable @{LogName='System'; Level=2; StartTime=$desde}).Count

---

Ejercicio 16. Exportar eventos a JSON

```
PS C:\Windows\System32> Get-WinEvent -LogName Application -MaxEvents 20 | ConvertTo-Json -Depth 4 | Out-File "C:\eventos.json"
PS C:\Windows\System32> cat C:\eventos.json
[
  {
    "Id": 1022,
    "Version": 0,
    "Qualifiers": 0,
    "Level": 2,
    "Task": 0,
    "Opcode": 0,
    "Keywords": 36028797018963968,
    "RecordId": 6180,
    "ProviderName": ".NET Runtime",
    "ProviderId": null,
    "LogName": "Application",
    "ProcessId": 0,
    "ThreadId": 0,
    "MachineName": "DESKTOP-M7CM9BE",
    "UserId": null,
    "TimeCreated": "2025-11-14T12:14:12.2936067+01:00",
```

Get-WinEvent -LogName Application -MaxEvents 20 | ConvertTo-Json -Depth 4 | Out-File "C:\Scripts\eventos.json"

- **Bloque 4: Automatización (17–20)**

Ejercicio 17. Crear una tarea diaria

Ejecuta C:\Scripts\resumen.ps1 todos los días a las 09:00.

$accion = New-ScheduledTaskAction -Execute "powershell.exe" -Argument "-File `"C:\resumen.ps1`""

$trigger = New-ScheduledTaskTrigger -Daily -At 09:00

Register-ScheduledTask -TaskName "InformeDiario" -Action $accion -Trigger $trigger

---

Ejercicio 18. Deshabilitar una tarea existente

Disable-ScheduledTask -TaskName "InformeDiario"

Ejercicio 19. Crear script de auditoría

Crea un script que guarde IP, procesos activos y hora actual en un TXT.

```
$ip = (Get-NetIPAddress -AddressFamily IPv4 | Where-Object {$_.IPAddress -notlike "169.*"} | Select-Object -First 1 -ExpandProperty IPAddress)

$procesos = (Get-Process | Measure-Object).Count

@"

Fecha: $(Get-Date)

IP: $ip

Procesos activos: $procesos

"@ | Out-File "C:\Scripts\auditoria.txt"
```

Ejercicio 20. Comprobar ejecución de una tarea

Muestra la última vez que se ejecutó la tarea “InformeDiario”.

```
Get-ScheduledTaskInfo -TaskName "InformeDiario" | Select LastRunTime, LastTaskResult
```